C Programming

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Running and Linking C Files

This section explains the basic workflow for creating and running C programs from the terminal. It covers writing a source file, compiling it into an executable using `gcc`, and executing the program to observe its output. Linking will be discussed in later steps when working with multiple source files or external libraries.

1.1 The `main` Function and Running a Program

This step introduces the `main` function as the entry point of every C program. It demonstrates how a minimal source file is compiled and executed from the command line. The example shows the structure of a valid C program and how output is produced using a standard library function.

```c
// Header file for input output functions
#include <stdio.h>
// Main function: entry point for execution
int main(void) {
    // Print a message to the terminal
    printf("Hello from main!\n");
    // Return value indicating successful execution
    return 0;
}
```

Compilation

The following command compiles the source file `hello.c` and produces an executable file named `hello`.

```
gcc -Wall hello.c -o hello
```

> `hello.c` specifies the input source file.
>
> `-o hello` sets the name of the output executable to `hello`.

Execution

The following command runs the executable from the current directory.

```
./hello
```

Output

```
Hello from main!
```

1.2 Review of core C concepts

This step uses a separate function to demonstrate how arrays are passed as pointers, how indexing and pointer arithmetic access the same memory, and how variable type sizes can be inspected using `sizeof`.

```c
// Header files for input output and fixed width integer types
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

// Function that prints array values using pointer access
void inspect_array(const int32_t *a, size_t n) {
    const int32_t *p = a;           /* pointer to first element */
    printf("Inside function: ");
    for (size_t i = 0; i < n; i++) {
        printf("%d", *(p + i));
        if (i + 1 < n) {
            printf(", ");
        }
    }
}

int main(void) {
    int32_t data[] = {10, 20, 30, 40};                /* array */
    size_t n = sizeof(data) / sizeof(data[0]);        /* number of elements */
    long vl = 42L;                                    /* long variable */

    printf("data (array base address)          : %p\n", (void*)data);
    printf("&data[0] (address of first elem)   : %p\n", (void*)&data[0]);
    printf("&data[1] (address of second elem) : %p\n", (void*)&data[1]);
    printf("data[2] = %d\n", data[2]);
    printf("sizeof(long)     = %zu\n", sizeof(long));
    printf("sizeof(int32_t) = %zu\n", sizeof(int32_t));
    inspect_array(data, n);

    return 0;
}
```

Code Details

When an array is passed to a function, the function parameter is a pointer to the first element of that array. This means the function can access and read the original data stored in memory, but it does not receive a separate copy of the array. The parameter `const int32_t *a` represents the starting address of the array in memory.

Array indexing provides a clear way to access elements using their position. The expression `a[i]` refers to the element at index `i` in the array, where indexing starts from zero.

Pointer arithmetic provides a lower-level way to work with memory addresses. The expres-

sion `p + i` computes the address of the `i`-th element by moving the pointer forward by `i` elements. The dereference operator `*` is then used to read the value stored at that address.

The output of `sizeof(long)` and `sizeof(int32_t)` shows how many bytes these variable types use on the current system. These values depend on the platform and architecture, which is why checking type sizes is important when writing portable code.

Compilation

```
gcc -Wall inspect_function.c -o InspectFunction
```

Execution

```
./InspectFunction
```

Output

```
data (array base address)         : 0x7ffc2a1b3c10
&data[0] (address of first elem)  : 0x7ffc2a1b3c10
&data[1] (address of second elem) : 0x7ffc2a1b3c14
data[2] = 30
sizeof(long)    = 8
sizeof(int32_t) = 4
Inside function: 10, 20, 30, 40
```

1.3 Static Linking

This part demonstrates how to split a program into a small library and a main file, then build and link them together using static linking. The function implementation is placed in a separate source file and compiled into a static library. The main program links against this library at build time.

In a standard project structure, a header file usually named `header.h` is used to store the function declarations that define the public interface of the library. Both the library source file and the main program include this file using the `#include` directive. This ensures that all source files share the same function signatures and prevents mismatches when the code is compiled.

File Layout

- `inspect.c`: Contains the function implementation that will be compiled into a static library.
- `main.c`: Contains the `main` function and a local declaration of the library function.

Library Source File: inspect.c

```c
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

// Function implementation
void inspect_array(const int32_t *a, size_t n) {
    const int32_t *p = a;

    printf("Inside function: ");
    for (size_t i = 0; i < n; i++) {
        printf("%d", *(p + i));
        if (i + 1 < n) {
            printf(", ");
        }
    }
    printf("\n");
}
```

Main Program: main.c

```c
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

// Function declaration, normally placed in a header
void inspect_array(const int32_t *a, size_t n);

int main(void) {
    int32_t data[] = {10, 20, 30, 40};
    size_t n = sizeof(data) / sizeof(data[0]);
    printf("data (array base address)          : %p\n", (void*)data);
    printf("&data[0] (address of first elem)   : %p\n", (void*)&data[0]);
    printf("&data[1] (address of second elem) : %p\n", (void*)&data[1]);
    printf("data[2] = %d\n", data[2]);
    inspect_array(data, n);
    printf("sizeof(long)     = %zu\n", sizeof(long));
    printf("sizeof(int32_t) = %zu\n", sizeof(int32_t));
    return 0;
}
```

Linking Multiple Object Files Directly

Before creating a static library archive, it is useful to see the simplest form of multi-file compilation. We can compile each source file into an object file and then link them directly into one executable.

First, compile both source files into object files:

```
gcc -Wall -c inspect.c -o inspect.o
gcc -Wall -c main.c -o main.o
```

Then, link the object files together:

```
gcc main.o inspect.o -o simple_app
```

In this case, the linker combines the object files directly into a single executable. No archive file is required.

Build Process Using a Static Library

Instead of linking object files directly, we can create a static library archive. This is useful when we want to collect multiple object files into a single reusable library.

First, compile the library source file into an object file:

```
gcc -Wall -c inspect.c -o inspect.o
```

Then, create a static library archive from the object file.

```
ar rcs libinspect.a inspect.o
```

> `ar` runs the archive utility that creates and maintains static library files.
>
> `r` tells `ar` to add the object file to the archive, replacing any existing version of the same file.
>
> `c` tells `ar` to create the archive file if it does not already exist.
>
> `s` tells `ar` to generate a symbol index so the linker can locate functions in the library efficiently.
>
> `libinspect.a` is the name of the static library archive that is produced.
>
> `inspect.o` is the compiled object file that is stored inside the archive.

Then, compile and link the main program with the static library.

```
gcc -Wall main.c -L. -linspect -o static_app
```

> `main.c` specifies the main source file to compile.
>
> `-L.` tells the linker to search the current directory for libraries.
>
> `-linspect` links against the library named `libinspect.a`.

> `-o static_app` sets the name of the output executable to `static_app`.

Execution

```
./static_app
```

Output

The output of this program is the same as in the previous step because static linking changes how the program is built, not how it behaves at runtime.

Note

In static linking, the object code from the library is copied directly into the final executable at link time, making the program self-contained and independent of the library file at runtime.

1.4 Dynamic Linking

This part uses the same source files shown in the Static Linking subsection. The library source file `inspect.c` and the main program `main.c` are unchanged. Only the build process differs, because the library is compiled as a shared object instead of a static archive.

In dynamic linking, the library remains a separate file and is loaded by the operating system when the program starts. The executable does not contain the library's machine code. Instead, it contains a reference to the shared library file and a list of symbols that must be resolved at runtime.

Inspecting the Statically Linked Executable

Before building the shared library, we can inspect the previously compiled executable `static_app` using the `ldd` command:

```
ldd ./static_app
```

This command lists the shared libraries required at runtime. Although `libinspect.a` was linked statically, the executable is not fully static because the `-static` option was not used.

`libc.so` appears because the program uses standard C library functions such as `printf`.

`ld-linux-x86-64.so` is the dynamic loader responsible for loading shared libraries and resolving symbols before execution begins.

A fully statically linked executable would require compilation with the `-static` option.

Build Process

First, compile the library source file into position-independent code and create a shared library.

```
gcc -Wall -fPIC -shared inspect.c -o libinspect.so
```

> `-fPIC` generates position-independent code so the library can be loaded at different memory addresses.
>
> `-shared` tells the compiler to produce a shared library instead of an executable.
>
> `libinspect.so` is the resulting shared library file.

Then, compile and link the main program against the shared library.

```
gcc -Wall main.c -L. -linspect -o dynamic_app
```

> `-L.` tells the linker to search the current directory for libraries.
>
> `-linspect` links against the shared library named `libinspect.so`.
>
> `-o dynamic_app` sets the name of the output executable.

Execution

Before running the program, ensure the runtime linker can find the shared library.

```
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH$
./dynamic_app
```

What Happens at Runtime

When a dynamically linked program starts, the following steps occur before the `main` function begins:

- The kernel loads the executable file into memory.
- Control is transferred to the dynamic loader `ld-linux-x86-64.so`.
- The dynamic loader reads the list of required shared libraries from the executable.
- It searches for these libraries in standard locations and loads them into memory.
- Symbol references are resolved and runtime initialization code is executed.
- The program's startup code is invoked, which then calls the `main` function.

Viewing Loaded Libraries

You can inspect which shared libraries a program will use with the following command:

```
ldd ./dynamic_app
```

This command prints a list of shared libraries and their locations. The output typically includes both the custom library and system libraries such as the C standard library.

Output

The output is the same as in the Static Linking subsection. Dynamic linking changes when and how the library code is loaded, not the program's behaviour.

2 Dynamic Memory Allocation and Memory Handling

This section introduces how programs request, use, and release memory at runtime. Unlike local variables and fixed-size arrays, dynamic memory lives on the heap and is managed explicitly by the programmer.

2.1 Requesting Memory with `malloc`

The function `malloc` asks the operating system for a block of memory of a given size.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
   int n = 4;
   int *a = (int *)malloc(n * sizeof(int));
   if (!a) {
      printf("Memory allocation failed");
      return 1;
   }
   for (int i = 0; i < n; i++) {
      a[i] = (i + 1) * 10;
   }
   for (int i = 0; i < n; i++) {
      printf("%d ", a[i]);
   }
   free(a);
   return 0;
}
```

Output

```
10 20 30 40
```

Explanation

`malloc` returns a pointer to the first byte of a block of memory on the heap. The pointer type determines how the program interprets that memory. In this example, the block is treated as an array of integers.

If `malloc` returns `NULL`, it means the operating system could not provide the requested memory. A program must always check for this before using the pointer.

Using `sizeof`

Memory is allocated in bytes, not in number of elements. When writing:

```
malloc(n * sizeof(int))
```

the program requests enough bytes to store `n` integers. Forgetting `sizeof` usually results in allocating too little memory, which leads to memory corruption.

2.2 Releasing Memory with `free`

Calling `free` tells the operating system that the program no longer needs a block of memory. After calling `free`, the pointer becomes invalid and must not be used again.

```c
#include <stdlib.h>

int main(void) {
   int *p = (int *)malloc(10 * sizeof(int));
   if (!p) return 1;
   free(p);
   return 0;
}
```

Failing to call `free` leads to memory leaks, where a program gradually consumes more memory over time.

2.3 Resizing Memory with `realloc`

Sometimes the required memory size is not known in advance. The function `realloc` allows an existing block to be resized.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
   int *a = (int *)malloc(2 * sizeof(int));
   if (!a) return 1;
   a[0] = 1;
   a[1] = 2;
   int *b = (int *)realloc(a, 4 * sizeof(int));
   if (!b) {
      free(a);
      return 1;
   }
   b[2] = 3;
   b[3] = 4;
   for (int i = 0; i < 4; i++) {
      printf("%d ", b[i]);
   }
   free(b);
   return 0;
}
```

Output

```
1 2 3 4
```

Explanation

`realloc` may return the same pointer or a new one. The return value must always be stored in a new variable first. If reallocation fails, the original memory block is still valid and must be freed.

2.4 Stack vs Heap

When a program runs, memory is divided into different regions.

Stack: Stores local variables and function call information. Memory is allocated and freed automatically when functions are entered and exited.

Heap: Stores dynamically allocated memory. Memory is requested from the operating system and must be released manually.

```c
#include <stdio.h>
#include <stdlib.h>

int *make_heap_value(void) {
   int *p = (int *)malloc(sizeof(int));
   if (!p) return NULL;
   *p = 99;
   return p;
}

int main(void) {
   int stack_value = 42;
   int *heap_value = make_heap_value();
   printf("Stack value: %d\n", stack_value);
   printf("Heap value: %d\n", *heap_value);
   free(heap_value);
   return 0;
}
```

Output

```
Stack value: 42
Heap value: 99
```

Explanation

The variable `stack_value` exists only inside `main`. When the function returns, its stack frame disappears automatically.

The pointer `heap_value` refers to memory allocated on the heap. This memory exists outside any function and remains valid until it is freed.

2.5 Copying Memory with memcpy

memcpy copies raw bytes, not values. It does not interpret the data type.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
   int src[4] = {10, 20, 30, 40};
   int *dst = (int *)malloc(4 * sizeof(int));
   if (!dst) return 1;
   memcpy(dst, src, 4 * sizeof(int));
   for (int i = 0; i < 4; i++) {
      printf("%d ", dst[i]);
   }
   free(dst);
   return 0;
}
```

Output

```
10 20 30 40
```

2.6 Setting Memory with memset

memset sets raw bytes, not typed values. It is commonly used to clear buffers before use.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
   int *a = (int *)malloc(4 * sizeof(int));
   if (!a) return 1;
   memset(a, 0, 4 * sizeof(int));
   for (int i = 0; i < 4; i++) {
      printf("%d ", a[i]);
   }
   free(a);
   return 0;
}
```

Output

```
0 0 0 0
```

3 Introduction to Reading from and Writing to Files

This part introduces how a program interacts with the operating system's file system. Instead of working only with memory, the program now moves data between disk storage and main

memory. In C, an open file is represented by a pointer of type `FILE *`. This pointer represents a connection between your program and a file managed by the operating system. The program does not access the disk directly; it requests the operating system to perform the operations.

3.1 Writing Data to a File (Text Mode)

This example shows how a program creates a file and writes human-readable text into it.

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("output.txt", "w");
    if (!f) {
        perror("fopen");
        return 1;
    }
    for (int i = 1; i <= 5; i++) {
        fprintf(f, "Value: %d\n", i);
    }
    fclose(f);
    return 0;
}
```

Opening a file is done with:

```c
FILE *f = fopen("output.txt", "w");
```

The mode `"w"` specifies write mode. The operating system creates the file if it does not exist, or clears its contents if it already exists. If the operation fails, `fopen` returns `NULL`.

Writing data is done with:

```c
fprintf(f, "Value: %d\n", i);
```

Closing the file is done with:

```c
fclose(f);
```

3.2 Reading Data from a File (Text Mode)

This example shows how a program reads values from a file into memory.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *f = fopen("output.txt", "r");
    if (!f) {
        perror("fopen");
        return 1;
    }
    int x;
```

```
    while (fscanf(f, "%d", &x) == 1) {
        printf("Read: %d\n", x);
    }
    fclose(f);
    return 0;
}
```

Opening a file for reading is done with:

```
FILE *f = fopen("output.txt", "r");
```

The mode `"r"` specifies read mode. The operating system opens the file and positions the file pointer at the beginning. If the file does not exist or cannot be opened, `fopen` returns `NULL`.

Reading data is done with:

```
fscanf(f, "%d", &x);
```

Each call requests the operating system to transfer data from the file on disk into memory. The variable resides in RAM, while the file resides on disk, and the OS handles moving the bytes between them.

The operating system maintains a file position indicator. Each read operation advances this position, so the next read continues from the following bytes in the file.

Closing the file is done with:

```
fclose(f);
```

3.3 Binary Files

Binary files store raw memory without text formatting. They preserve the exact byte layout of data, which makes them useful for systems programming. However, the program must know exactly how many bytes to read and how to interpret them.

Reading Raw Bytes from a File

The following program reads a file byte by byte and prints each byte in hexadecimal form. The file is opened in binary mode so that the program reads the exact bytes stored on disk.

```
#include <stdio.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <file>\n", argv[0]);
        return 1;
    }
    FILE *f = fopen(argv[1], "rb");
    if (!f) {
        perror("fopen");
```

```
        return 1;
    }
    unsigned char byte;
    while (fread(&byte, 1, 1, f) == 1) {
        printf("%02X ", byte);
    }
    fclose(f);
    return 0;
}
```

Opening a file in binary read mode is done with:

```
FILE *f = fopen(argv[1], "rb");
```

The mode `"rb"` means read in binary mode. In this mode, the program reads the exact bytes from the file without any text interpretation or conversion.

Binary mode can be used to read almost any type of file, including text files, images, executables, or other binary data, because the program treats the file as a sequence of raw bytes.

The function `fread` returns the number of elements successfully read. If this number is smaller than expected, it usually indicates the end of the file or an error.

Compile and use the program to inspect a text file:

```
gcc read_bytes.c -o read_bytes
./read_bytes output.txt
```

Example output:

```
56 61 6C 75 65 3A 20 31 0A 56 61 6C 75 65 3A 20 32 0A ...
```

This prints the raw byte contents of the text file.

Writing a Binary File

This program writes raw integer values directly to a binary file.

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    int32_t values[] = {10, 20, 30, 40};
    FILE *f = fopen("data.bin", "wb");
    if (!f) {
        perror("fopen");
        return 1;
    }
    fwrite(values, sizeof(int32_t), 4, f);
    fclose(f);
    return 0;
}
```

The call

```
fwrite(values, sizeof(int32_t), 4, f);
```

writes four 32-bit integers from memory to the file. The parameters specify the starting address, the size of each element, the number of elements, and the file handle.

The function `fwrite` returns the number of elements successfully written. A value smaller than the requested count indicates an error.

Inspecting the Binary File

First, compile and run the writing program to create the binary file:

```
gcc write_bin.c -o write_bin
./write_bin
```

Then inspect the created file using the byte-reading program:

```
./read_bytes data.bin
```

For example, the number 10 (hexadecimal `0x0000000A`) appears in memory as:

```
0A 00 00 00
```

Note that due to the little-endian representation (used in the x86 architecture), the least significant byte of a multi-byte data value is stored at the lowest memory address.

This demonstrates that binary files store the exact byte representation of values as they exist in memory.

[Comment: Maybe add a note here that on a POSIX (including Linux) OS, there is absolute no difference between opening with "rb" (or "wb") and "r" (or "w"), but that one Windows there are differences due to end-of-line handling and EOF handling.]